

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Домашнее задание 5

**Реализация межсервисного взаимодействия посредством
очередей сообщений**

Выполнил:
Цатинян Артём
Группа БР1.2

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

1 Задача

Тема проекта: приложение для бронирования столиков в ресторанах.

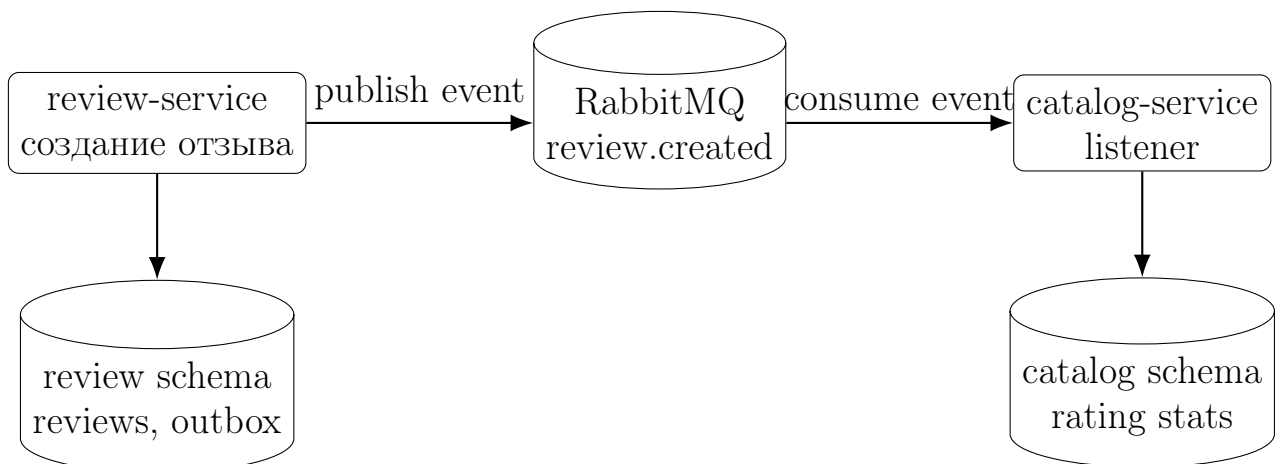
В рамках домашнего задания требовалось подключить брокер сообщений RabbitMQ или Kafka и реализовать межсервисное взаимодействие через очередь сообщений. Для текущего проекта был выбран RabbitMQ, так как он хорошо подходит для событийной интеграции между Spring Boot сервисами и не требует избыточной инфраструктуры для учебного сценария.

2 Ход работы

К началу работы приложение уже было разделено на микросервисы: `identity-service`, `catalog-service`, `booking-service` и `review-service`. Синхронное взаимодействие между сервисами использовалось для операций, где результат нужен сразу, например проверка пользователя или получение контекста бронирования. Для ДЗ5 был добавлен асинхронный сценарий, в котором сервис отзывов сообщает каталогу о создании нового отзыва.

2.1 Выбор сценария

В качестве бизнес-сценария выбрано обновление рейтинга ресторана после публикации отзыва. Это хороший пример для очереди сообщений, потому что пользовательский запрос на создание отзыва не должен напрямую зависеть от успешности пересчета агрегированного рейтинга в каталоге. Отзыв создается в `review-service`, после чего событие `ReviewCreated` публикуется в RabbitMQ. `catalog-service` получает это событие и обновляет таблицу агрегированной статистики рейтинга.



2.2 Настройка RabbitMQ

RabbitMQ подключен как инфраструктурный сервис. Для локального запуска был добавлен файл `docker-compose.rabbitmq.yml`, который поднимает брокер и management UI.

```
rabbitmq:
  image: rabbitmq:3.13-management
  ports:
    - "5672:5672"
    - "15672:15672"
  environment:
    RABBITMQ_DEFAULT_USER: storage
    RABBITMQ_DEFAULT_PASS: storage
```

Для сервисов добавлены зависимости Spring AMQP. Параметры подключения вынесены в `application.yml`: `host`, `port`, `username` и `password`. В Docker Compose для ЛРЗ эти параметры переопределяются так, чтобы сервисы обращались к брокеру по имени контейнера `rabbitmq`.

2.3 Контракт события

Общий контракт события вынесен в модуль `common`. Это позволяет producer и consumer использовать один и тот же DTO и одинаковые имена `exchange`, `queue` и `routing key`.

Элемент	Значение
Exchange	<code>restaurant.review.events</code>
Queue	<code>catalog.review-created</code>
Routing key	<code>review.created</code>
Событие	<code>ReviewCreatedEvent</code>
Producer	<code>review-service</code>
Consumer	<code>catalog-service</code>

Событие содержит минимальный набор данных, необходимый каталогу для обновления рейтинга:

```
data class ReviewCreatedEvent(
  val eventId: String,
  val reviewId: Long,
  val bookingId: Long,
  val restaurantId: Long,
  val userId: Long,
  val rating: Int,
  val occurredAt: LocalDateTime
```

)

2.4 Публикация события

В `review-service` создание отзыва и запись сообщения в `outbox` выполняются в рамках одной бизнес-операции. Для этого в схему `review` добавлена таблица `review_outbox`. Такой подход снижает риск потери события: если отзыв успешно сохранен, событие также остается в базе и может быть опубликовано позже.

Публикация реализована отдельным компонентом `ReviewOutboxPublisher`. Он периодически выбирает необработанные записи `outbox`, преобразует `payload` в `ReviewCreatedEvent`, отправляет событие в `RabbitMQ` через `RabbitTemplate` и после успешной отправки помечает запись как обработанную.

```
rabbitTemplate.convertAndSend(  
    ReviewEvents.EXCHANGE,  
    ReviewEvents.CREATED_ROUTING_KEY,  
    event  
)
```

2.5 Обработка события

В `catalog-service` добавлен listener `ReviewCreatedEventListener`. Он подписан на очередь `catalog.review-created` через аннотацию `@RabbitListener`. После получения события сервис вызывает метод репозитория `applyReviewCreatedEvent`.

В каталоге добавлены две таблицы:

Таблица	Назначение
<code>restaurant_rating_stats</code>	Хранит агрегированный рейтинг ресторана и количество отзывов. Эти данные используются в списке ресторанов и карточке ресторана.
<code>processed_review_events</code>	Хранит идентификаторы уже обработанных событий, чтобы повторная доставка из очереди не увеличивала рейтинг дважды.

Метод обработки сначала пытается записать `eventId` в таблицу `processed_review_events`. Если событие уже было обработано, вставка не выполняется и пересчет рейтинга пропускается. Если событие новое, сервис

блокирует текущую строку статистики ресторана, пересчитывает средний рейтинг и увеличивает счетчик отзывов.

2.6 Идемпотентность и надежность

Очереди сообщений обычно допускают повторную доставку события. Поэтому обработчик в `catalog-service` сделан идемпотентным: ключом идемпотентности является `eventId`. Даже если RabbitMQ повторно доставит то же сообщение, таблица `processed_review_events` не позволит применить его второй раз.

Также используется `outbox`-подход на стороне `review-service`. Он отделяет сохранение бизнес-данных от отправки сообщения в брокер. Если RabbitMQ временно недоступен, отзыв остается в базе, `outbox`-запись остается необработанной, и `publisher` попытается отправить событие повторно при следующем запуске.

2.7 Проверка результата

Для проверки сценария можно поднять RabbitMQ:

```
docker-compose -f docker-compose.rabbitmq.yml up -d
```

Затем запускаются сервисы `identity-service`, `catalog-service`, `booking-service` и `review-service`. После авторизации пользователь создает отзыв через `review-service`. В результате:

1. В схеме `review` появляется запись в `reviews`.
2. В `review_outbox` появляется событие, которое затем помечается как обработанное.
3. В RabbitMQ публикуется сообщение `ReviewCreatedEvent`.
4. `catalog-service` получает событие из очереди.
5. В схеме `catalog` обновляется `restaurant_rating_stats`.

Проверить инфраструктуру RabbitMQ можно через web-интерфейс:

```
http://localhost:15672  
login: storage  
password: storage
```

3 Вывод

В рамках домашнего задания было реализовано асинхронное межсервисное взаимодействие через RabbitMQ. Сервис отзывов стал `producer` событий о создании отзывов, а сервис каталога стал `consumer` этих событий и

обновляет агрегированный рейтинг ресторанов. Для надежности добавлен outbox-подход, а для защиты от повторной доставки сообщений реализована идемпотентная обработка через таблицу обработанных событий.

В результате микросервисная архитектура стала менее связанной: создание отзыва не зависит напрямую от синхронного вызова каталога, а данные рейтинга обновляются через событийную интеграцию.